IoT Firmware Updates

Architecture and security, following RFC 9019

Developed in assistance with Claude.

We acknowledge funding provided by Microelectronic Commons and the support of OUSW(R&E).

Simple solution

- Device asks for updates.
- Server sends latest firmware and version number.
- Check if the version number is higher, if yes, install.

Standard update mechanism



Stakeholders

Author

Creates the firmware image and signs the manifest. Usually the device vendor.

Vendor’s firmware team

Device Operator

Runs a fleet of devices day to day and decides when updates roll out.

Energy utility managing installed thermostats

Network Operator

Runs the network devices connect through; may limit or schedule update traffic.

Home ISP or LPWAN provider

Key Terms: Update Artifacts

Firmware image

The binary the device runs: its complete software, or one component of it.

A 380 KB image for a thermostat’s microcontroller

Manifest

Signed metadata about the image: what it is, who made it, which devices it is for.

“Version 24, for model T-200, SHA-256 digest a3f9…”

Trust anchor

A public key, plus associated data, that the device uses to verify signatures.

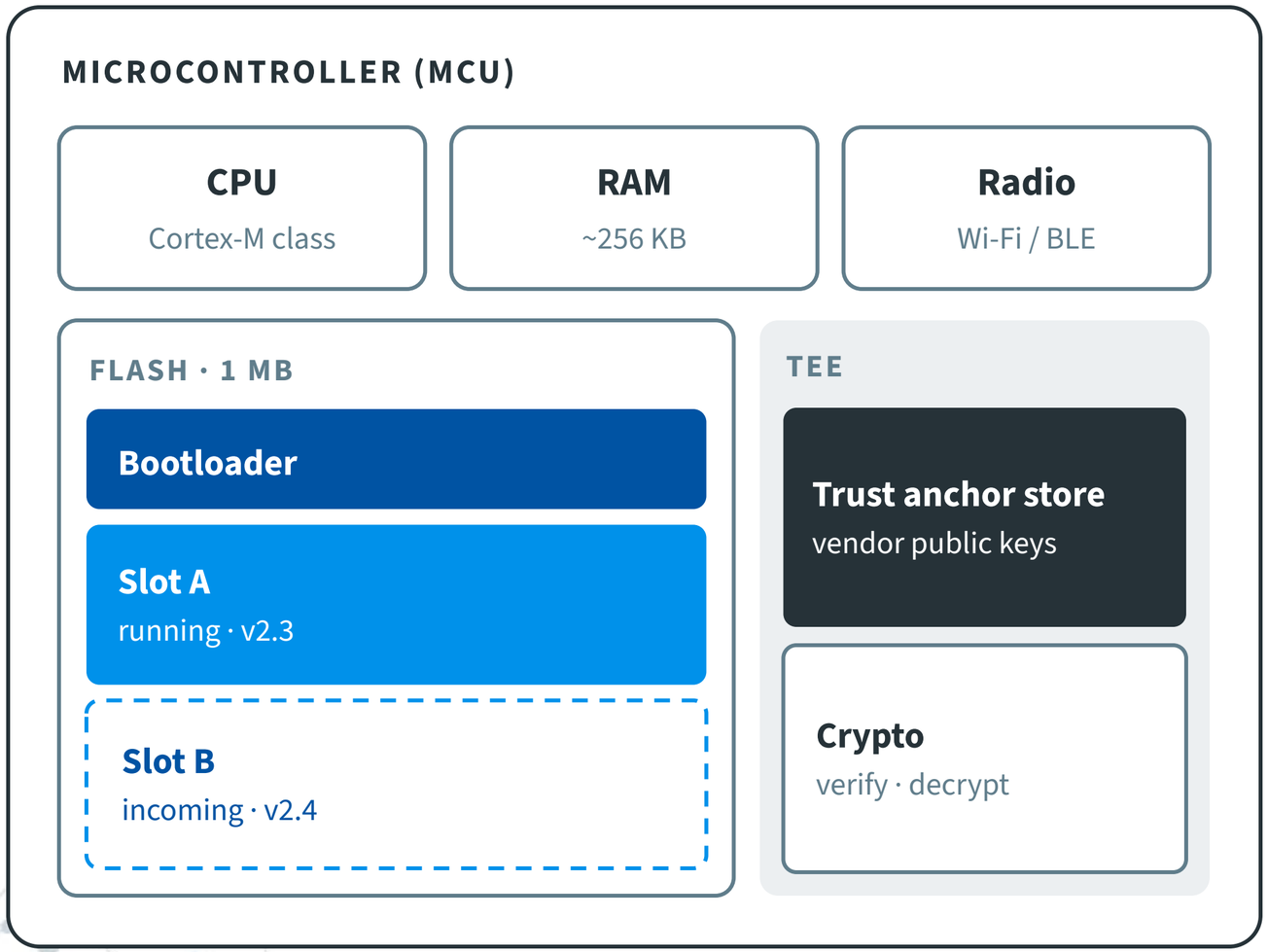
The vendor’s public key, written to the device at the factory

Anatomy of a Manifest

MANIFEST FIELD	QUESTION IT ANSWERS
`sequence-number: 24`	Is it newer than what I run?
`vendor-id: 3c1e…a7`	Is it from my vendor?
`class-id: t200-rev-b`	Is it built for my hardware?
`image-digest: sha256:a3f9…`	Is the image intact?
`image-size: 389120`	Do I have room for it?
`uri: fw.example.com/t200/24`	Where do I get it?
`dependencies: wifi-drv ≥ 1.8`	What else must be present?
`signature: ES256 (COSE_Sign1)`	**Who authorized all of the above?**

Information model: RFC 9124 · Encoding: IETF SUIT manifest (CBOR + COSE)

Key Terms: Device Components



Microcontroller

One chip with CPU, RAM, flash and peripherals; often only KBs of RAM

Bootloader

First code after reset; decides which image to run and checks it

Trust anchor store

Protected storage for the keys that verify updates

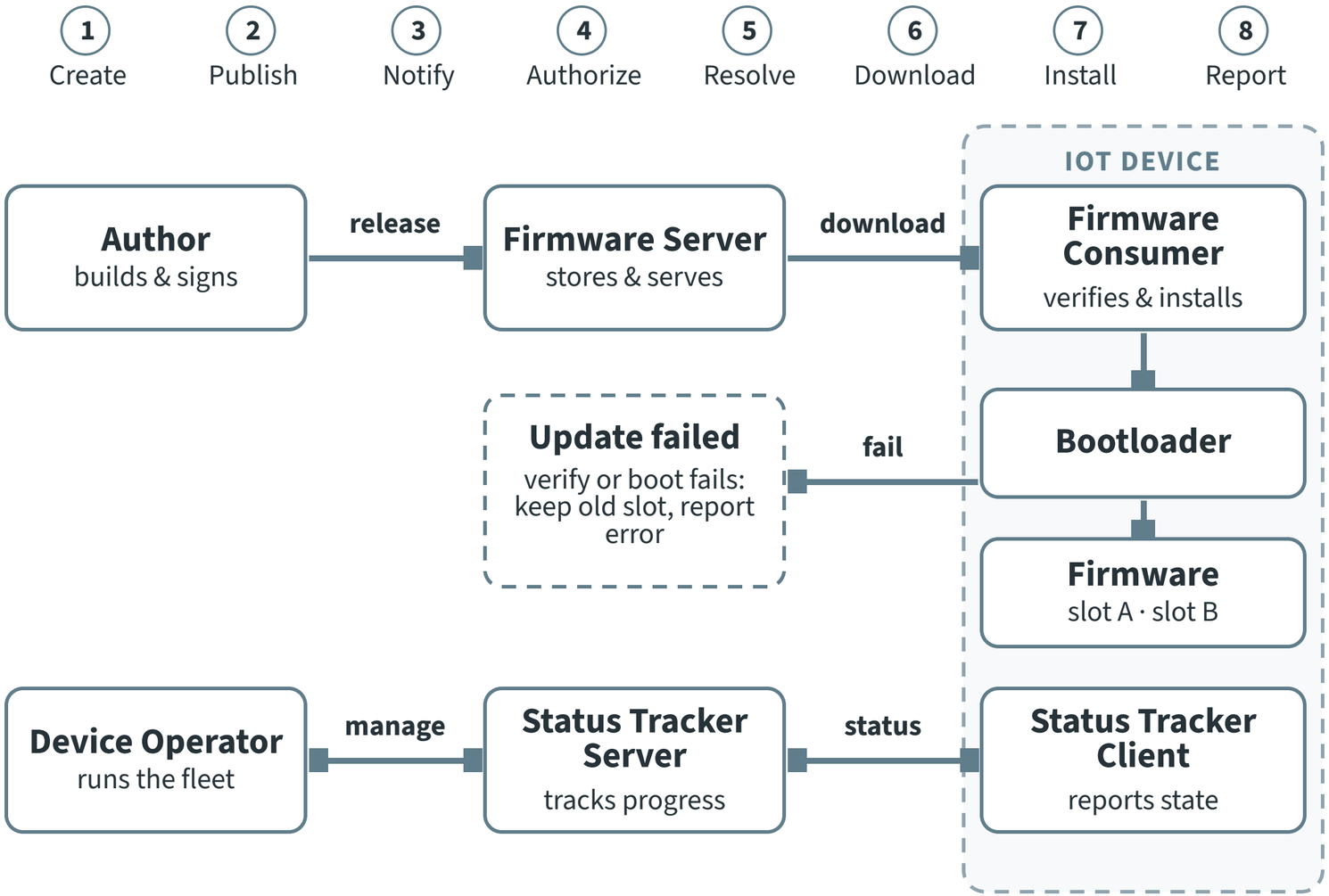
TEE

Trusted Execution Environment: isolates keys and crypto from normal code

Functional Entities

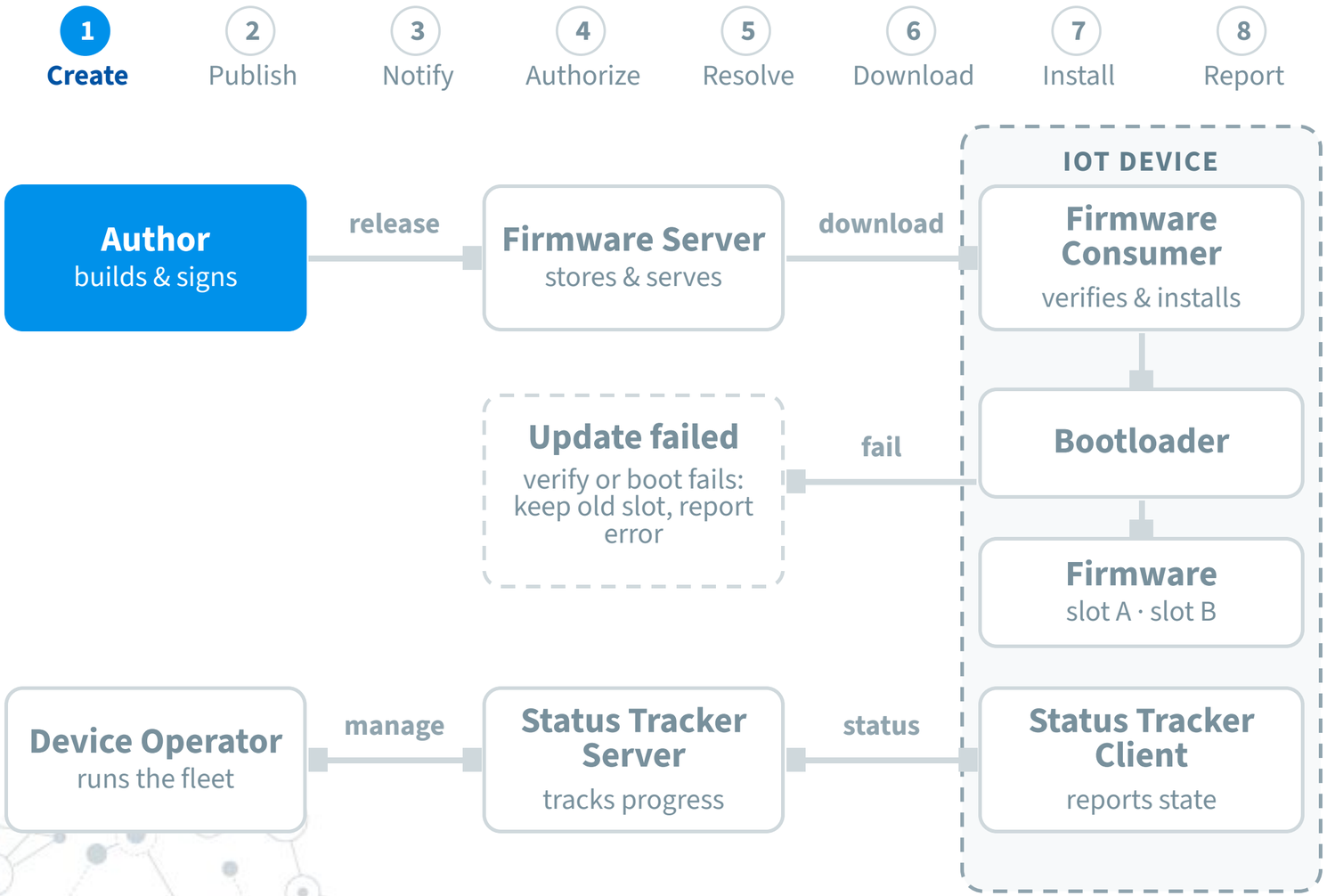
ENTITY	RUNS ON	RESPONSIBILITY
Firmware Consumer	Device	Fetches manifest and image, verifies both, installs the update
Status Tracker Client	Device	Reports device state, installed versions and update progress
Status Tracker Server	Back end	Collects status, triggers updates; often part of device management
Firmware Server	Back end / CDN	Stores firmware images and manifests for download

Firmware Update Architecture



Based on RFC 9019, Section 2

Step 1: Creation

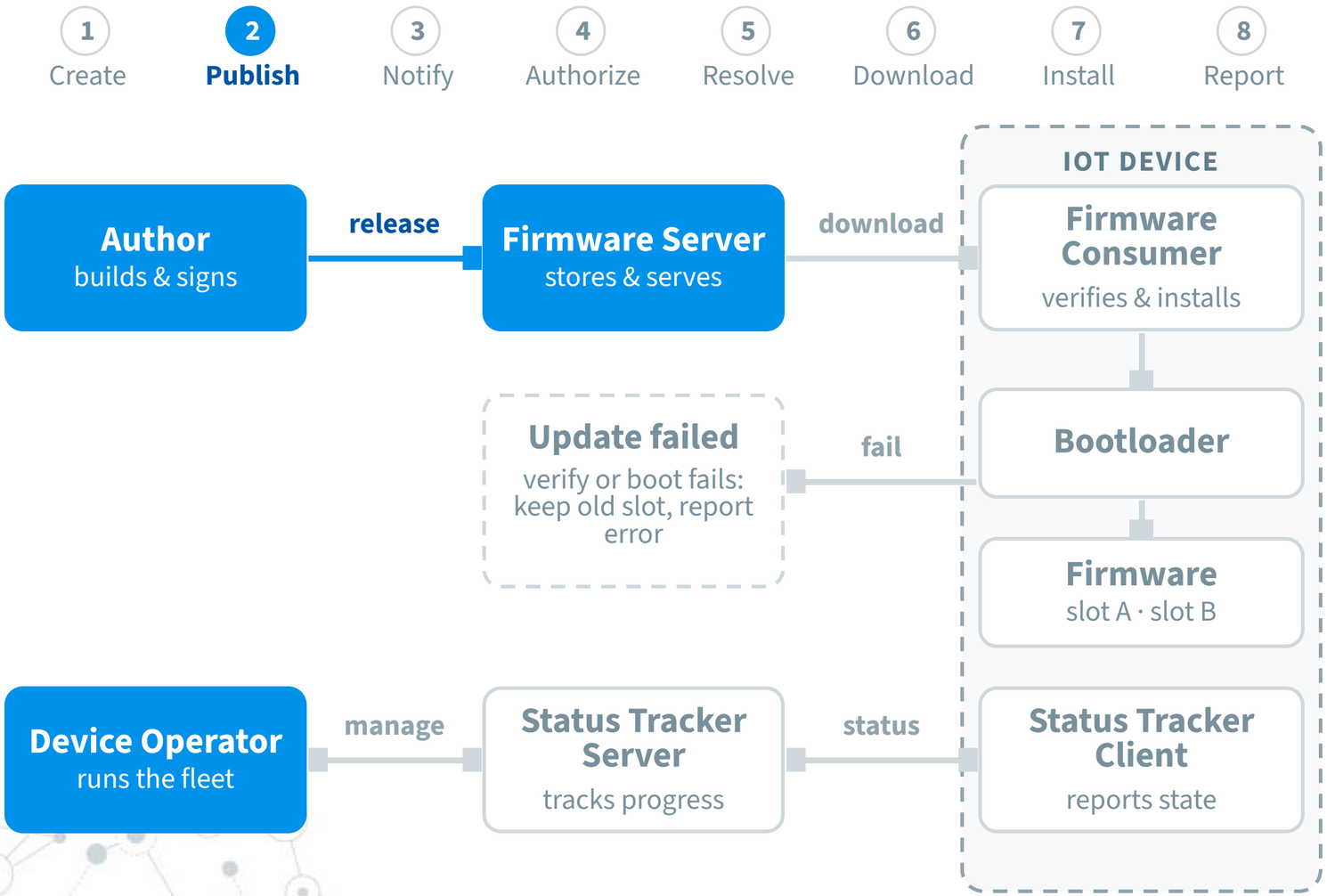


- Author builds the new firmware image
- Writes a manifest: version, target IDs, digest, location
- Signs the manifest; optionally encrypts the image

THERMOSTAT EXAMPLE

Vendor builds v2.4 and signs its manifest

Step 2: Publication

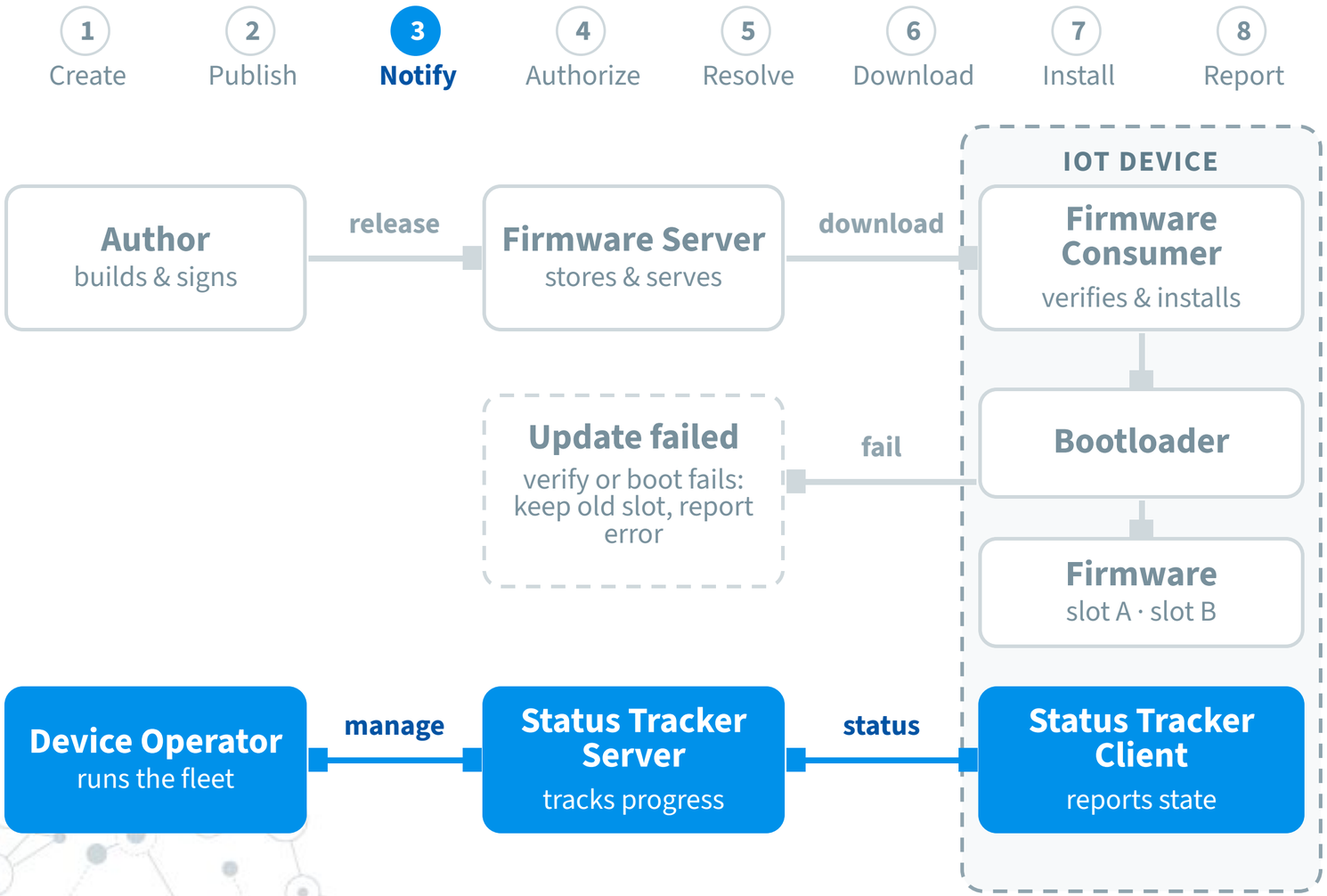


- Author releases image and manifest to the device operator
- Operator authorizes the release and uploads it to the firmware server
- Server may be a CDN; it only stores and serves, and need not be trusted

THERMOSTAT EXAMPLE

Vendor releases v2.4; the utility publishes it to its update server

Step 3: Notification



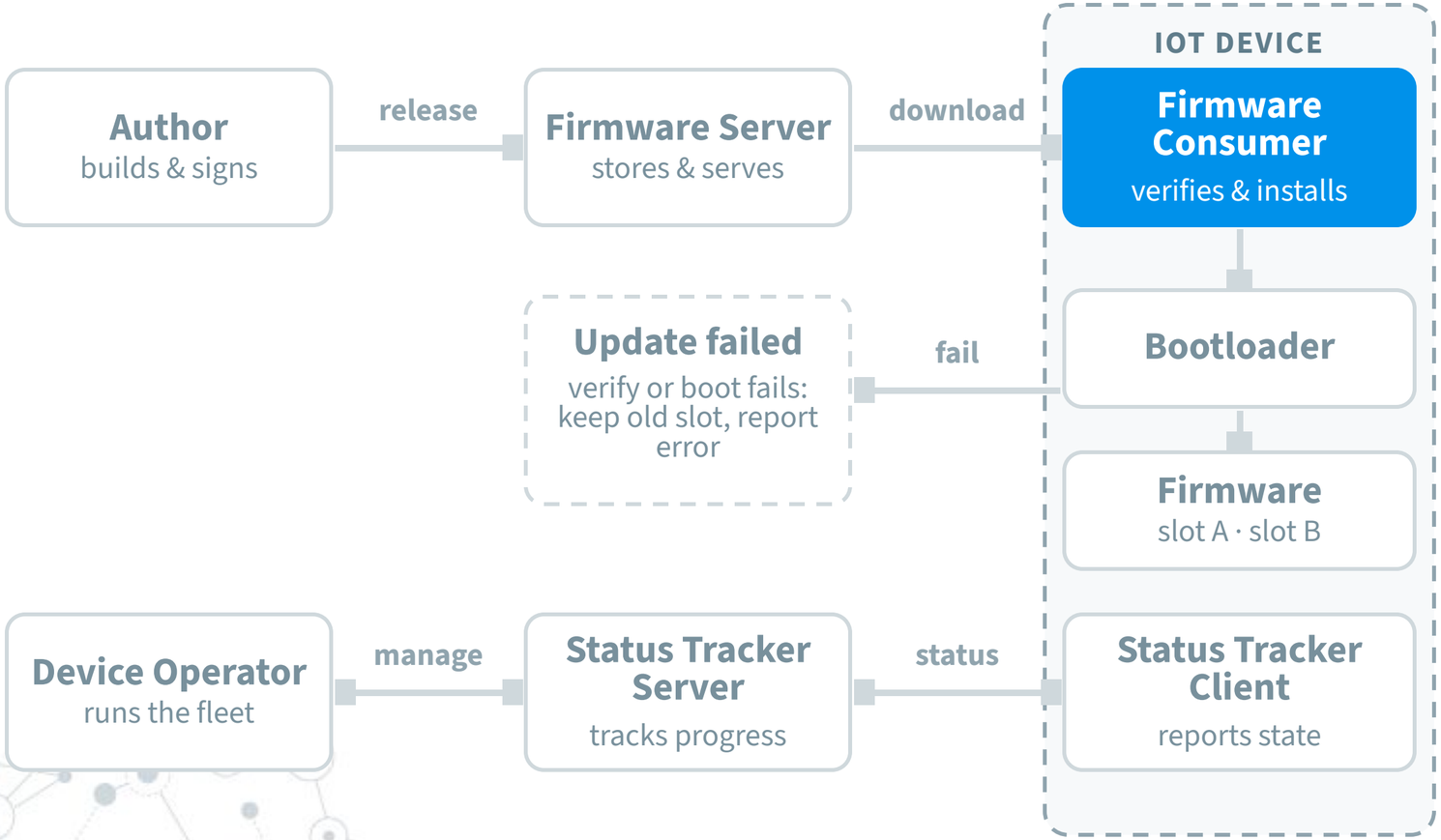
- Device learns that a new version exists
- Push: the server tells the device. Pull: the device polls
- Operator decides which devices update, and when

THERMOSTAT EXAMPLE

Utility schedules the rollout for 2 a.m., in batches

Step 4: Pre-authorization

1 Create · 2 Publish · 3 Notify · **4 Authorize** · 5 Resolve · 6 Download · 7 Install · 8 Report

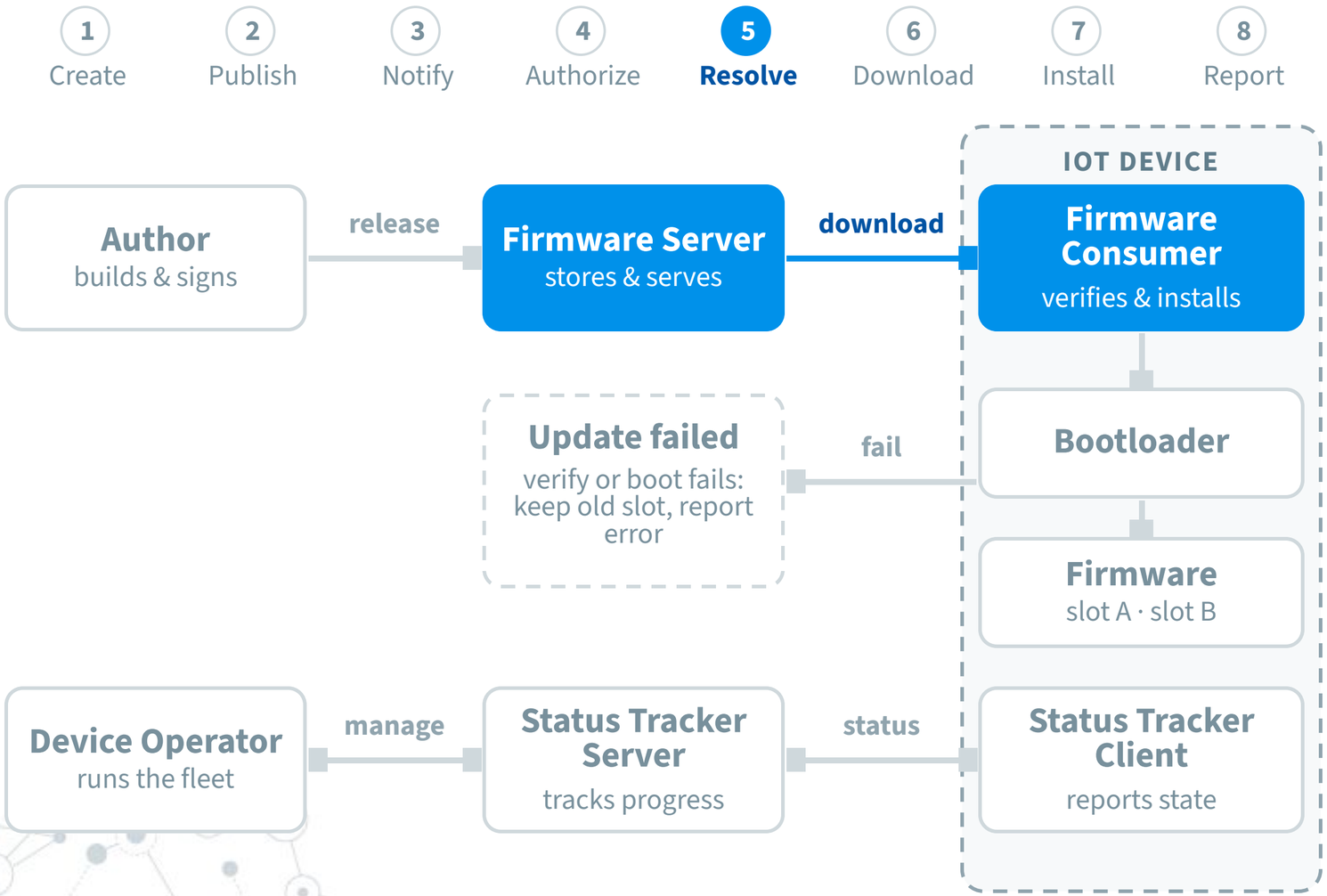


- Device fetches only the manifest first
- Verifies its signature with the trust anchor
- Checks IDs, version and local policy, e.g. battery level

THERMOSTAT EXAMPLE

Signed by the vendor, built for T-200, newer than v2.3: accept

Step 5: Dependency Resolution

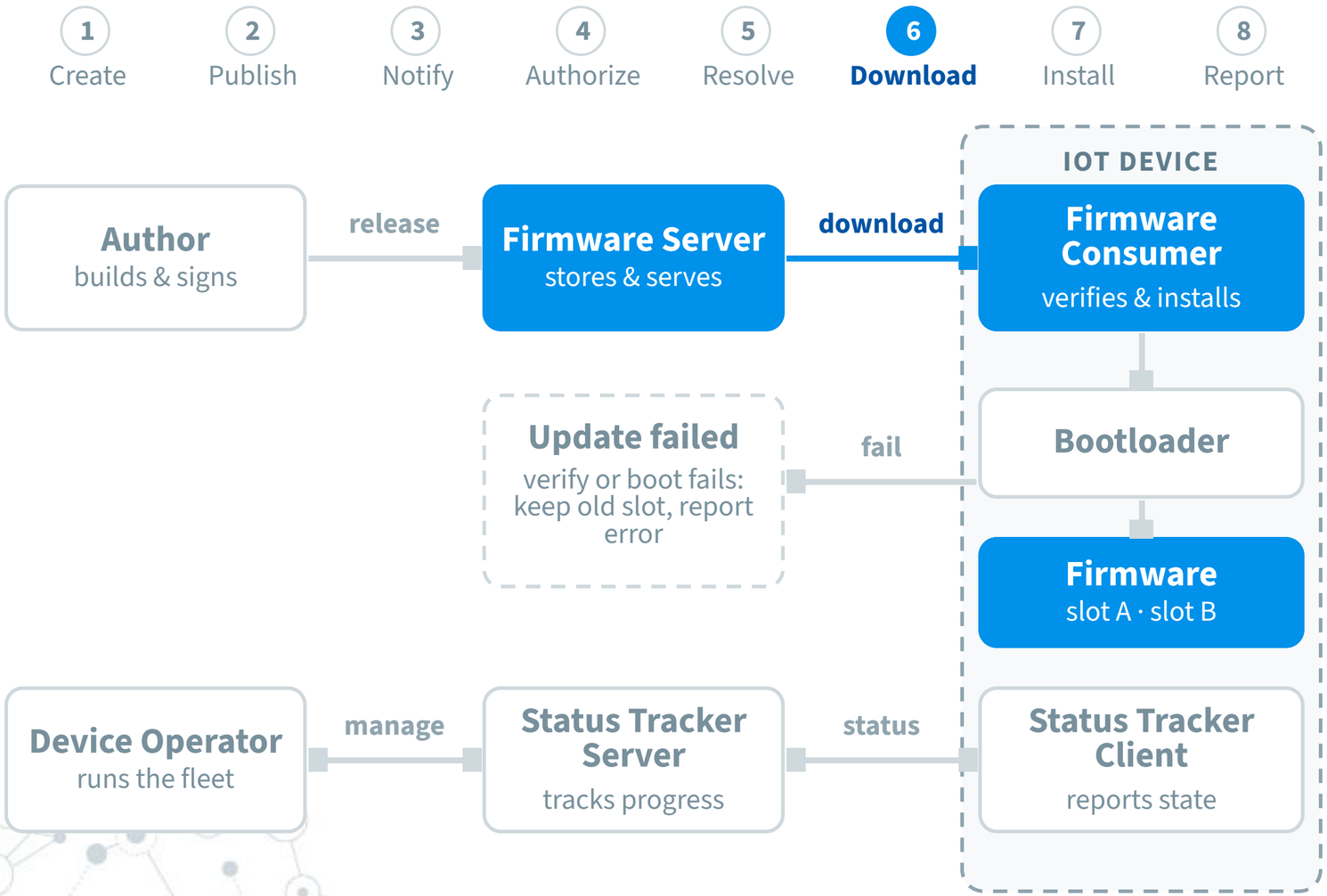


- Manifest may require other components or versions
- Device checks free storage and prerequisites
- Missing parts are fetched via further manifests

THERMOSTAT EXAMPLE

v2.4 needs Wi-Fi driver 1.8 or later; device has 1.9

Step 6: Download

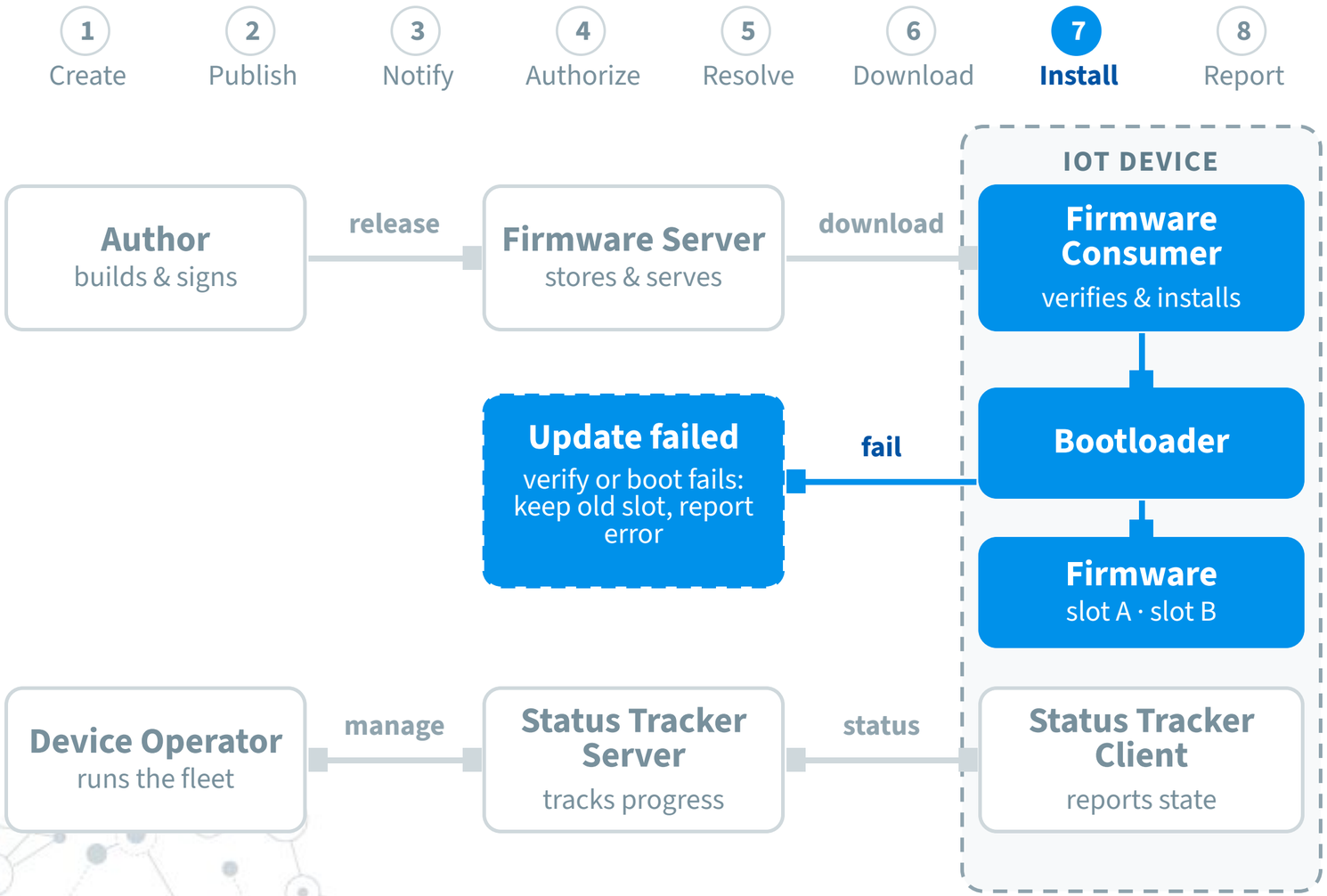


- Image fetched from the URI in the manifest
- On slow links: chunked, resumable or differential
- Stored in a free slot, never over the running code

THERMOSTAT EXAMPLE

380 KB image streamed into flash slot B

Step 7: Installation and Boot

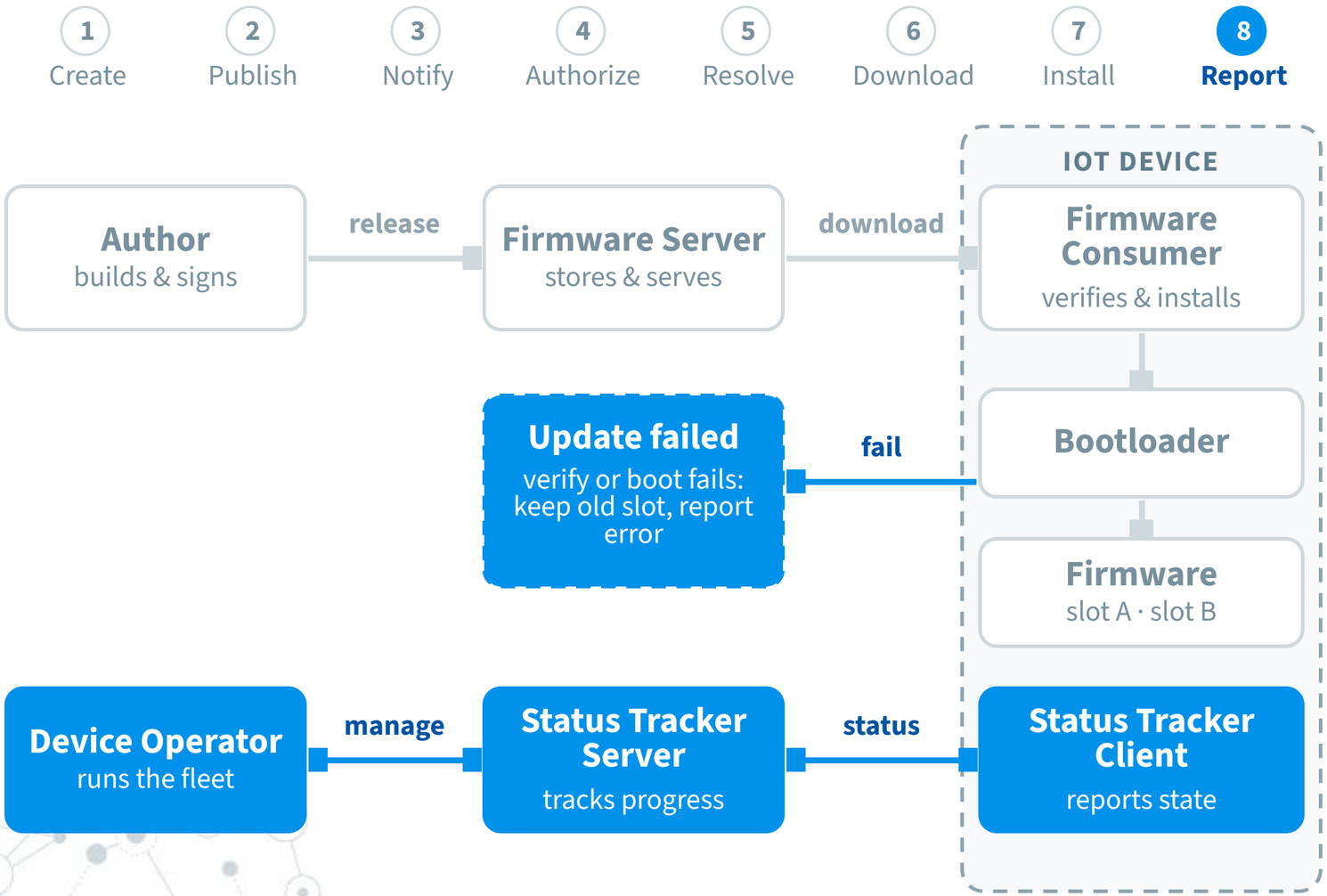


- Image digest checked against the manifest
- Device reboots; the bootloader verifies the image again
- On failure, it falls back to the previous image

THERMOSTAT EXAMPLE

Bootloader validates slot B and starts v2.4

Step 8: Status Reporting



- Device reports the result and its new version
- Operator monitors progress across the fleet
- Failed or stalled devices are retried or investigated

THERMOSTAT EXAMPLE

Dashboard lists devices still on v2.3 for retry

Device Requirements

Communication §1, §3

- Resumable download with flow and congestion control
- Status tracker client
- Discovery of tracker and firmware server

Storage §3, §4.1, §6

- Flash to write the new image
- Room for old + new image, or a recovery image
- Persistent sequence number

Crypto §3, §4.1

- Manifest parser
- Hash plus signature or MAC verification
- Decryption, if images are encrypted

Trust §2.1, §3, §7

- Tamper-resistant trust anchor store
- Anchors provisioned before any update
- Keys isolated in a TEE or secure element

Secure boot §4.1

- Bootloader verifies image before running it
- Recovery if the new image is invalid
- Exposes boot status to the status tracker

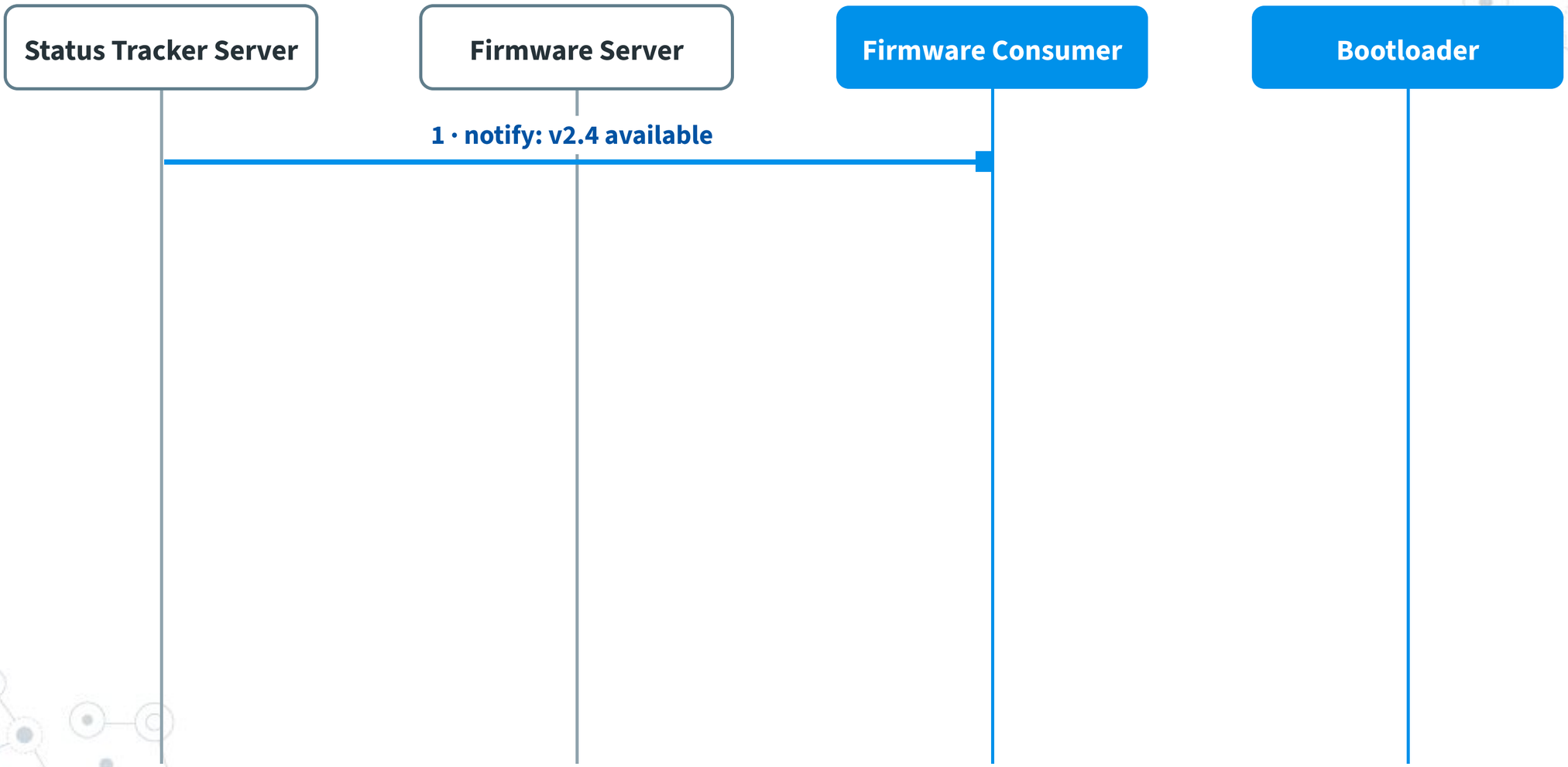
Robustness §1, §3, §4.1

- Survives power loss or network drop mid-update
- Updates unattended, no user interface
- Optional: boot measurements for attestation

Targets Class 1 devices (~10 KB RAM, ~100 KB flash) and up · symmetric keys allowed for the most constrained

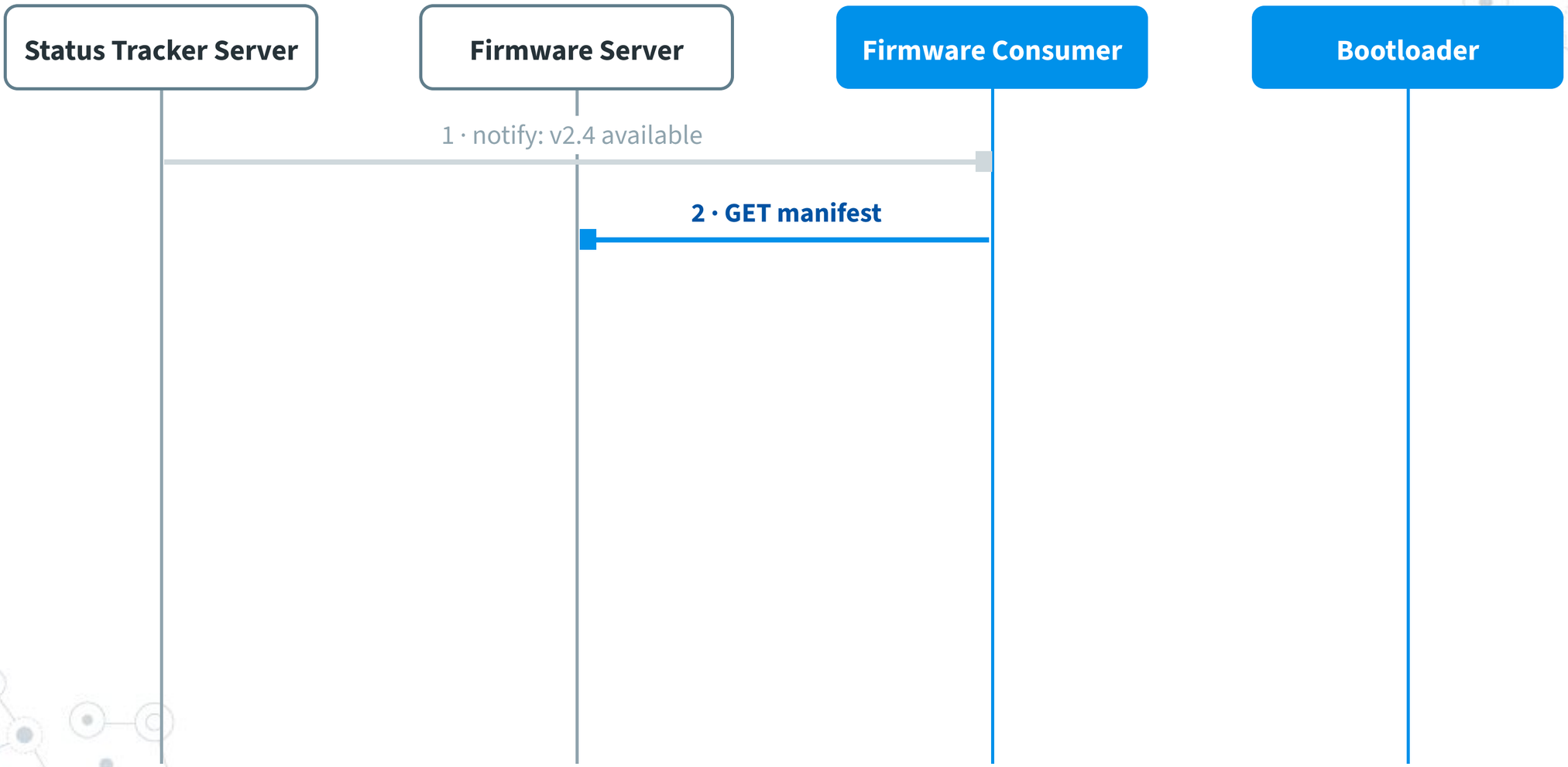
Example 1: Smart Thermostat

Single MCU · Wi-Fi · A/B flash slots · signed manifest



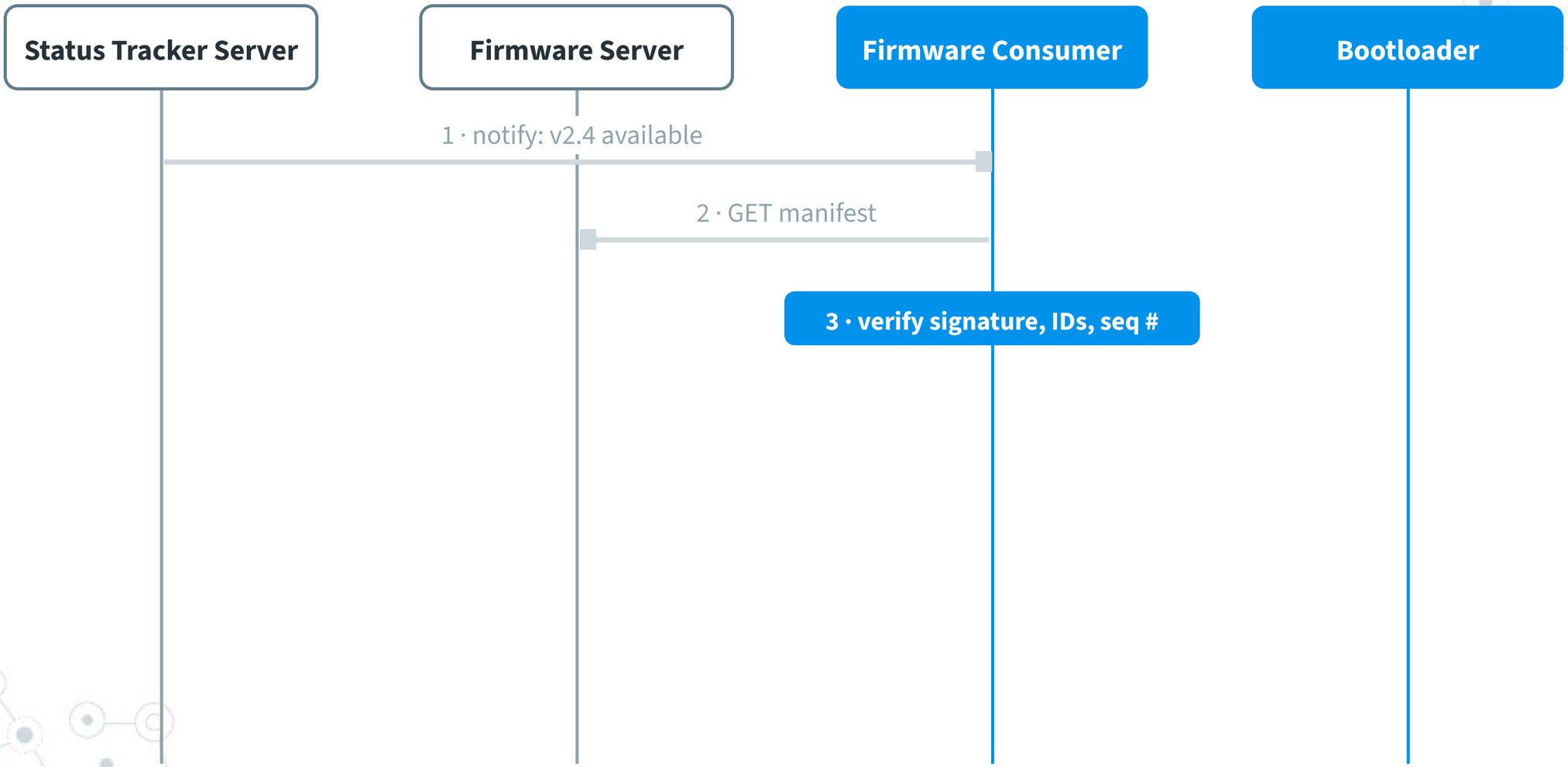
Example 1: Smart Thermostat

Single MCU · Wi-Fi · A/B flash slots · signed manifest



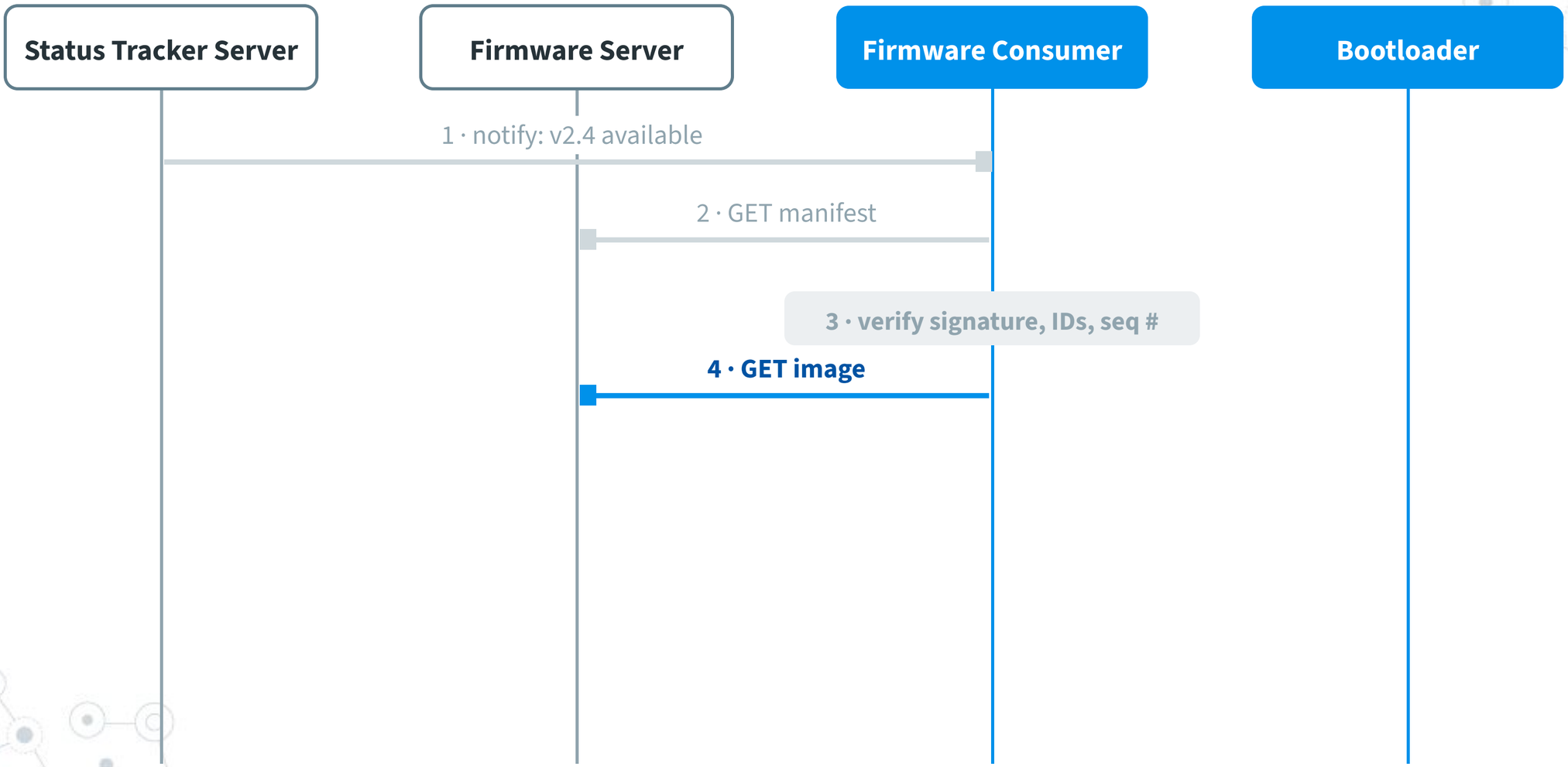
Example 1: Smart Thermostat

Single MCU · Wi-Fi · A/B flash slots · signed manifest



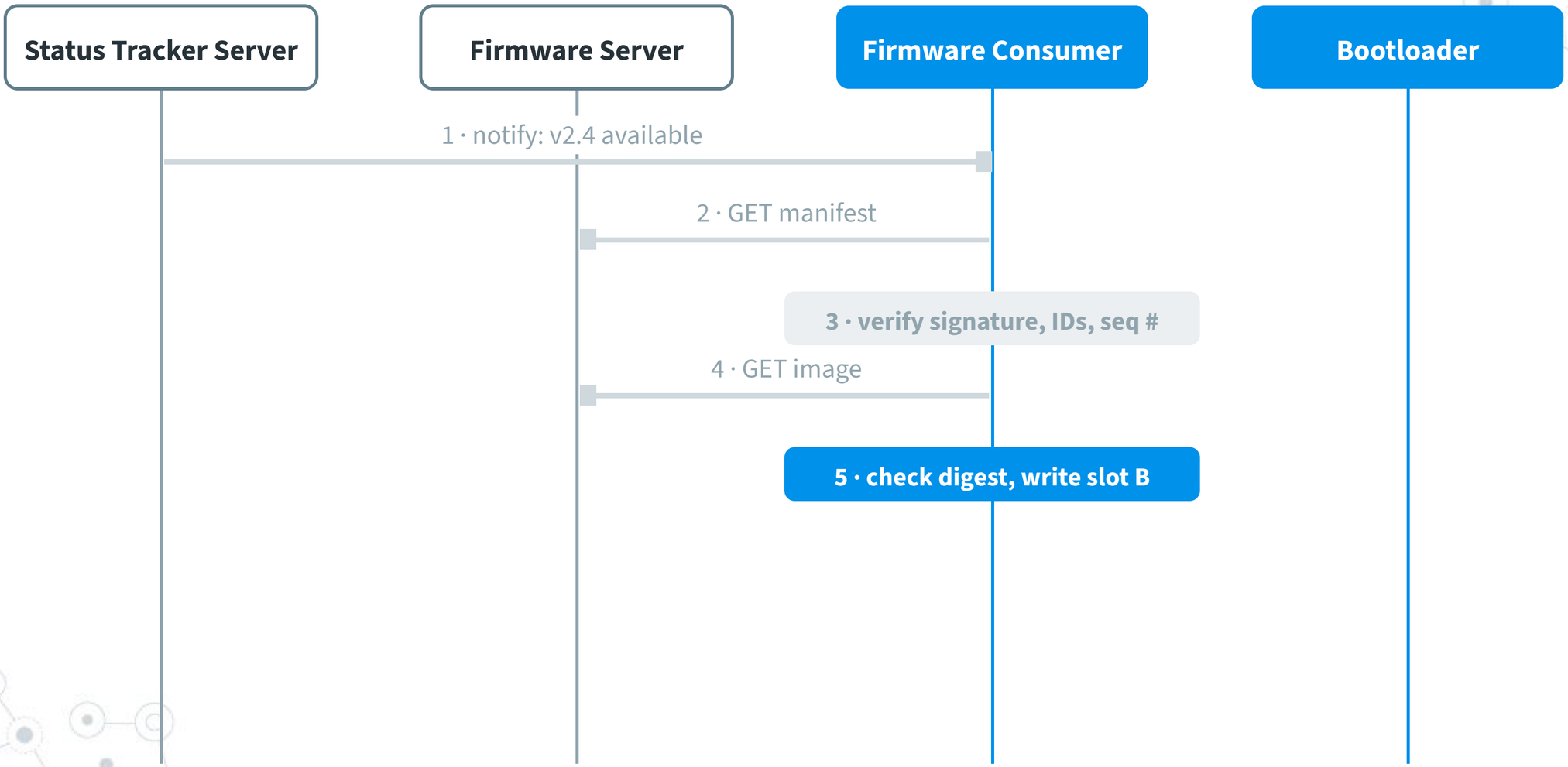
Example 1: Smart Thermostat

Single MCU · Wi-Fi · A/B flash slots · signed manifest



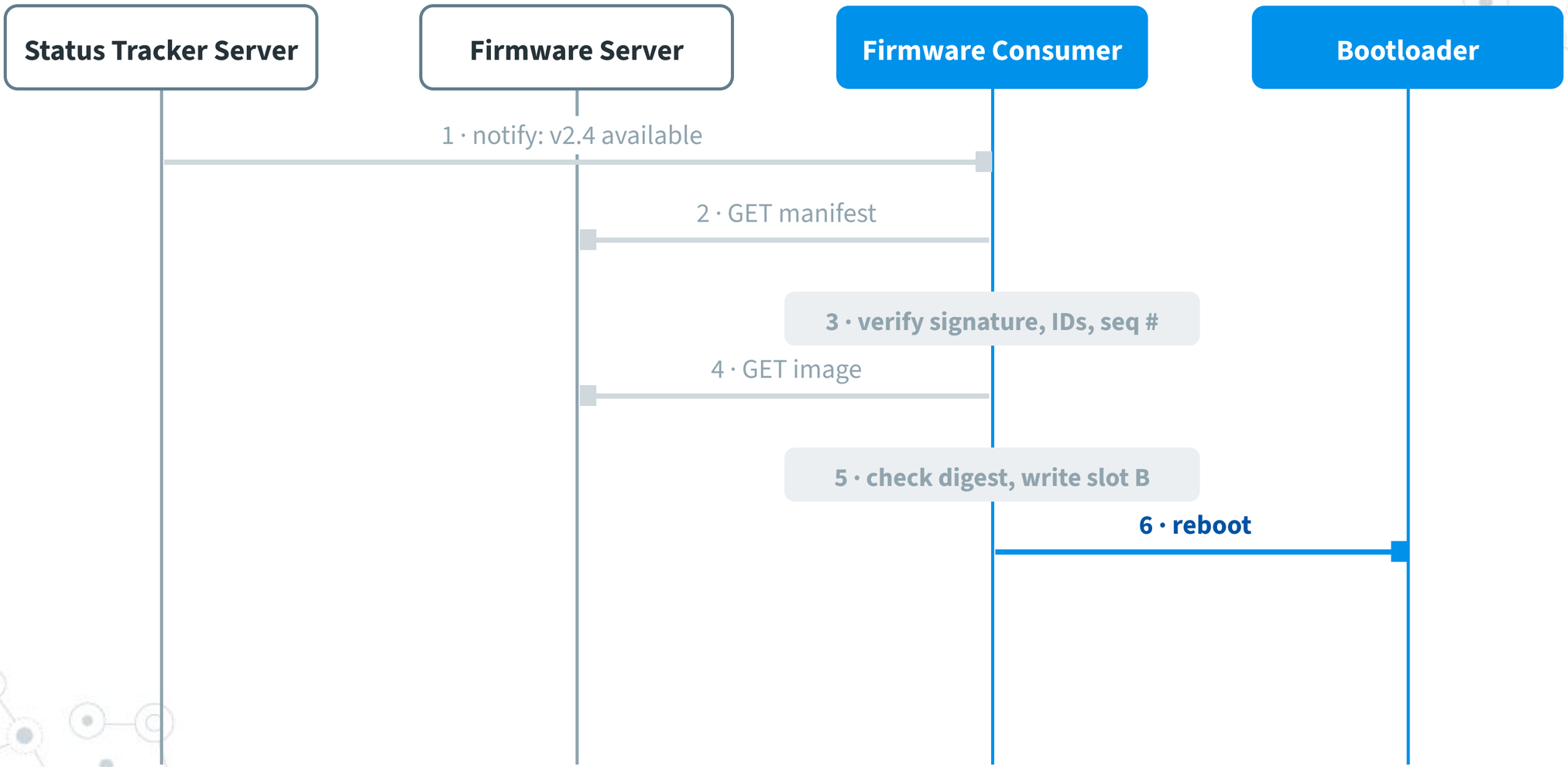
Example 1: Smart Thermostat

Single MCU · Wi-Fi · A/B flash slots · signed manifest



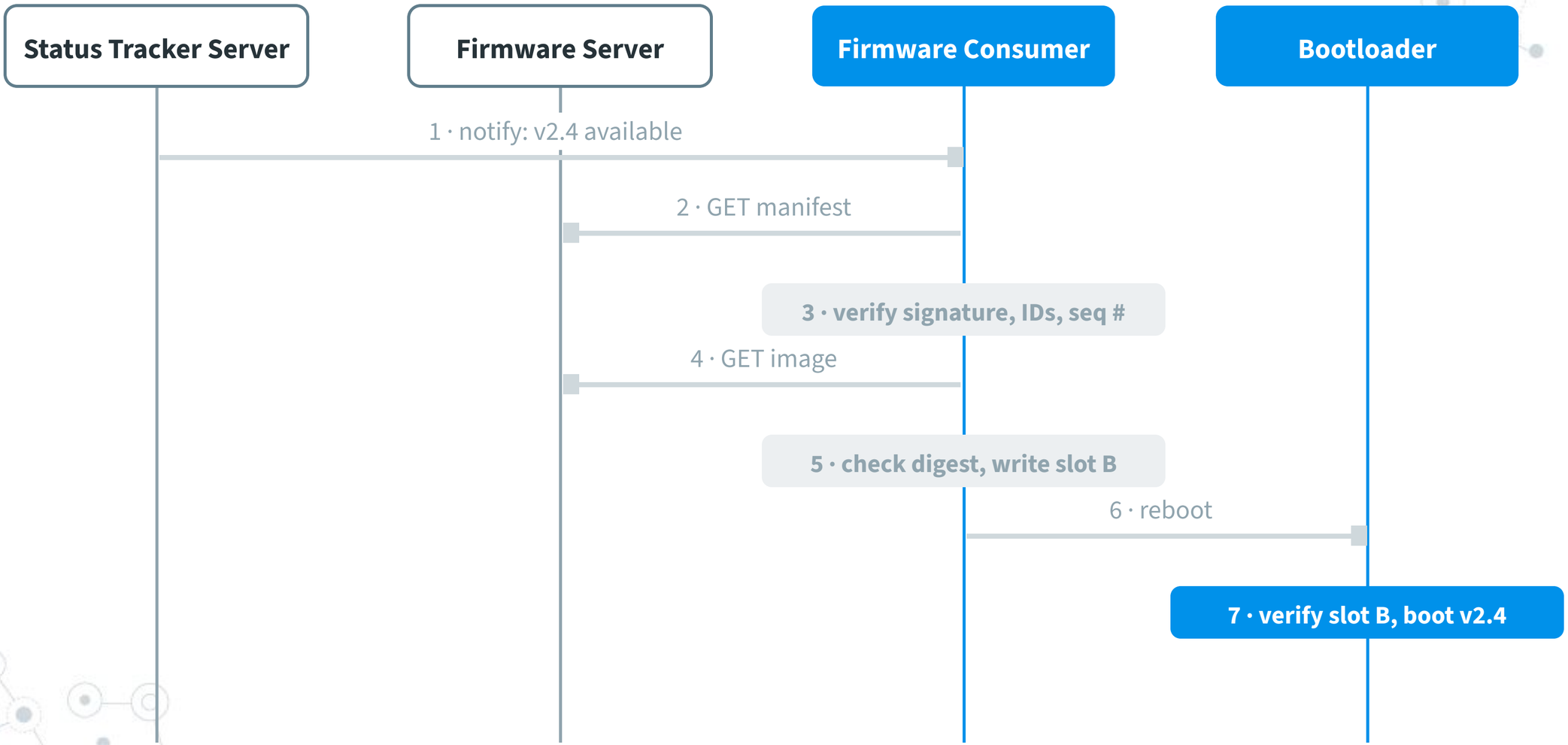
Example 1: Smart Thermostat

Single MCU · Wi-Fi · A/B flash slots · signed manifest



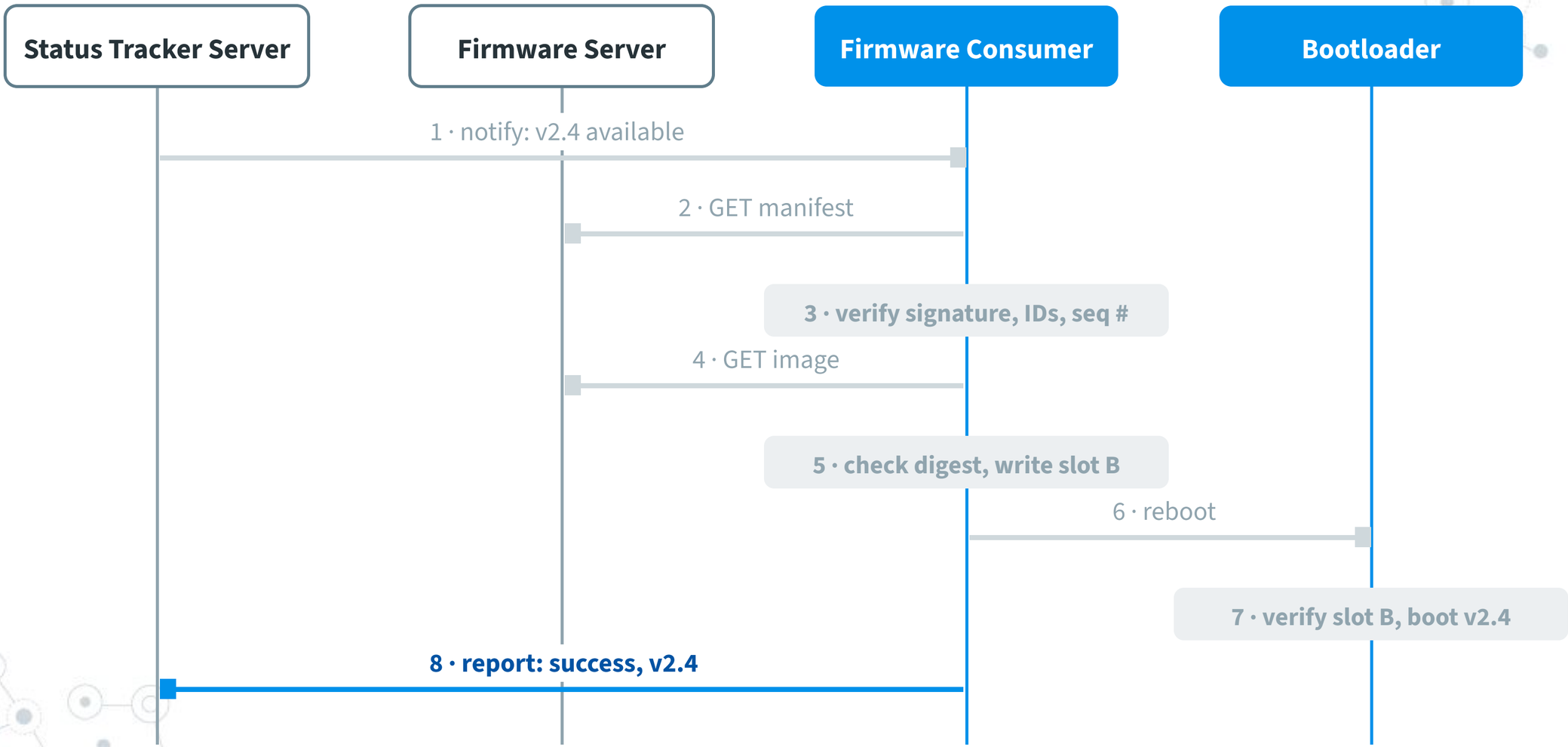
Example 1: Smart Thermostat

Single MCU · Wi-Fi · A/B flash slots · signed manifest



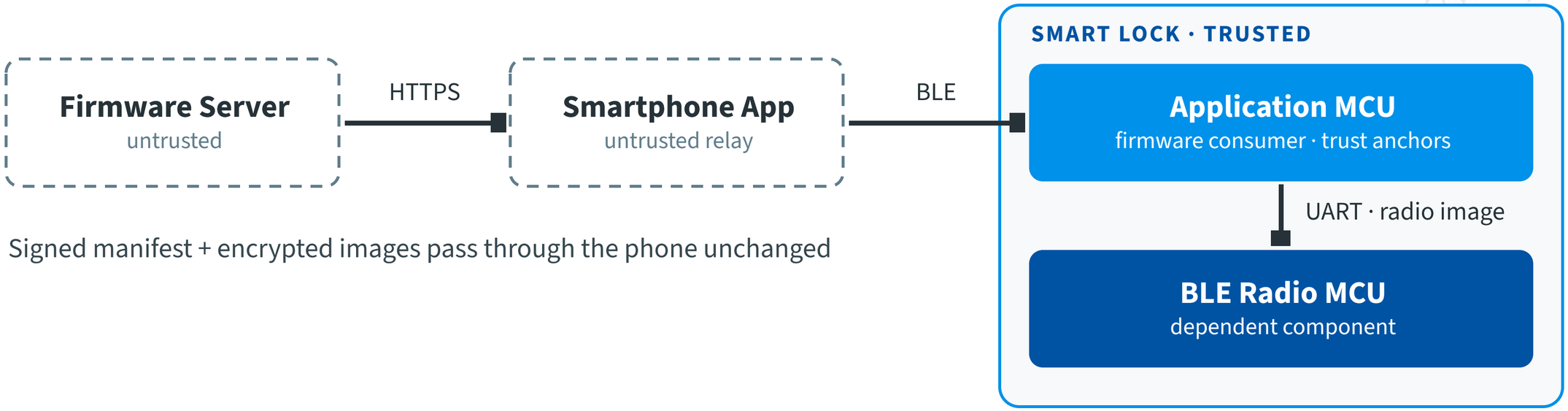
Example 1: Smart Thermostat

Single MCU · Wi-Fi · A/B flash slots · signed manifest



Example 2: Smart Lock

Two MCUs · BLE · phone as untrusted relay · encrypted images



Signed manifest + encrypted images pass through the phone unchanged

1. Phone downloads the update; it can neither read nor modify it
2. Application MCU verifies the signature, then decrypts with its device key
3. Dependency in manifest: radio firmware 3.1 is installed first
4. Each MCU's bootloader verifies its own image; status returns via the phone

Threats to General Updates

Threats to General Updates

T1 Tampered image

Image modified in transit or on the server, e.g. malware injected

T2 Unauthorized source

Attacker builds their own update and pushes it to devices

T3 Rollback and replay

An old, genuinely signed but vulnerable version is reinstalled

T4 Mismatched firmware

Valid image for another model is installed and bricks the device

T5 Firmware disclosure

Image intercepted and reverse engineered for flaws or IP

T6 Failed installation

Power loss or a bad image mid-update leaves the device unusable

T7 Compromised server

Attacker controls the firmware server, CDN or network path

T8 Key compromise

Signing key stolen, or a device's trust anchor replaced

T9 Resource exhaustion

Bogus updates drain battery, bandwidth or flash write cycles

Required Protection per Threat

THREAT	WHAT WE NEED	MECHANISM
T1 · Tampered image		
T2 · Unauthorized source		
T3 · Rollback and replay		
T4 · Mismatched firmware		
T5 · Firmware disclosure		
T6 · Failed installation		
T7 · Compromised server		
T8 · Key compromise		
T9 · Resource exhaustion		

Required Protection per Threat

THREAT	WHAT WE NEED	MECHANISM
T1 · Tampered image	**Integrity**	Image digest inside the signed manifest
T2 · Unauthorized source	**Authenticity**	Signature verified against a trust anchor
T3 · Rollback and replay	**Freshness**	Monotonic sequence number, stored on device
T4 · Mismatched firmware	**Applicability**	Vendor ID and class ID checked before install
T5 · Firmware disclosure	**Confidentiality**	Image encrypted end to end for the device
T6 · Failed installation	**Robustness**	A/B slots, verify before boot, fallback image
T7 · Compromised server	**End-to-end security**	Author signs; server and network stay untrusted
T8 · Key compromise	**Key protection**	Signing key in an HSM; trust anchors in protected storage
T9 · Resource exhaustion	**Early rejection**	Pre-authorize the manifest before downloading

Secure Firmware Update

TRUSTED · VENDOR

Author
signs & encrypts

Signs manifest with a key held in an HSM

T2 T8

Puts digest, sequence number, vendor and class IDs in manifest

T1 T3 T4

Encrypts the image for target devices

T5

UNTRUSTED · CLOUD

Firmware Server
CDN · network

Stores signed manifest and encrypted image

Any tampering here is detected on the device

T7

TLS/DTLS adds transport protection, but security does not depend on it

TRUSTED · DEVICE

Firmware Consumer
verifies before use

Verifies manifest signature with the trust anchor

T2

Checks IDs and sequence number before downloading

T3 T4 T9

Checks digest, decrypts, writes to the inactive slot

T1 T5

TRUSTED · DEVICE

Bootloader
secure boot

Re-verifies the image before every boot

T1 T2

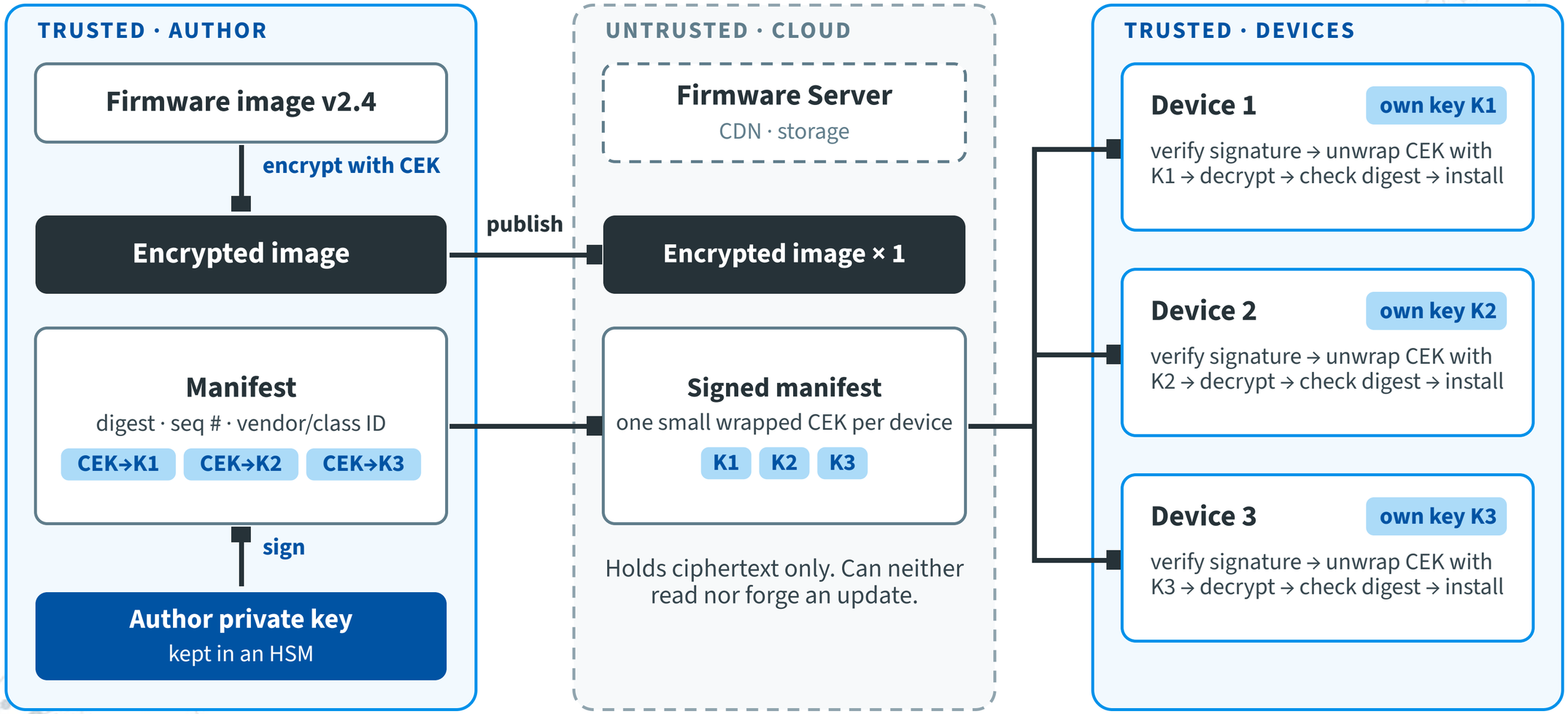
Keeps the previous image as a fallback

T6

Reads trust anchors from protected storage

T8

Secure Update: Sign and Encrypt



Same encrypted image for every device; only the wrapped CEK differs

To Think/Design choices:

Is Manifest encrypted? Should we encrypt it?
Advantages/Disadvantages

Per-device encryption v/s group encryption.